



Use GIT for Linux Kernel Development

This article is aimed at newbie developers who are planning to set up a development environment or move their Linux kernel development environment to GIT.

GIT is a free, open source distributed version control tool. It is easy to learn and is also fast, as most of the operations are performed locally. It has a very small footprint. Just to compare GIT with another SVN (Source Version Control) tool, refer to <http://git-scm.com/about/small-and-fast>.

GIT allows multiple local copies (branches), each totally different from the other—it allows the making of clones of the entire repository so each user will have a full backup of the main repository. Figure 1 gives one among the many pictorial representations of GIT. Developers can clone the main repository, maintain their own local copy (branch and branch1) and push the code changes (branch1) to the main repository. For more information on GIT, refer to <http://git-scm.com/book>.



Note: GIT is under development and hence changes are often pushed into GIT repositories. To get the latest GIT code, use the following command:
`$git clone git://git.kernel.org/pub/scm/git/git.git`

The kernel

The kernel is the lowest level program that manages

communications between the software and hardware using IPC and system calls. It resides in the main memory (RAM), when any operating system is loaded in memory.

The kernel is mainly of two types - the micro kernel and the monolithic kernel. The Linux kernel is monolithic, as is depicted clearly in Figure 2.

Based on the above diagram, the kernel can be viewed as a resource manager; the managed resource could be a process, hardware, memory or storage devices. More details about the internals of the Linux kernel can be found at <http://kernelnewbies.org/LinuxVersions> and <https://www.kernel.org/doc/Documentation/>.

Linux kernel files and modules

In Ubuntu, kernel files are stored under the `/boot/` directory (run `ls /boot/` from the command prompt). Inside this directory, the kernel file will look something like this:

```
'vmlinuz-A.B.C-D'
```

... where A,B is 3.2, C is your version and D is a patch or fix. Let's delve deeper into certain aspects depicted in Figure 3:

- `Vmlinux-3.2.0-29-generic`: In `vmlinuz`, 'z' indicates the